Required Capstone Checkpoint 7.1: Production Readiness – Security and Performance Optimization

Directions

Download this worksheet and save a copy to your device before you begin working. Use this worksheet to document the security safeguards and performance optimizations you implement for your capstone system. Type your responses directly into the document, save your completed work, and upload the finished file when you are ready to submit.

Completion Checklist

☒ Implemented one or more safeguards that improve system security and/or reliability
☒ Implemented one or more strategies that improve system performance or efficiency
☒ Provided evidence demonstrating the impact of the implemented changes
☒ Evaluated trade-offs among security, performance, cost, latency, and output quality
☒ Reflected on remaining limitations and future improvements

Step 1. System Overview

- Briefly describe your current capstone system.
- Summarize the task your system is designed to complete.

Current system overview
My capstone is a Wikipedia question-answering system built with a graph-based RAG workflow. It decomposes a question, retrieves relevant articles, selects sections and tables, performs table calculations when needed, checks whether the evidence is sufficient, and writes an answer grounded in the retrieved material. The final version also sanitizes input, checks answer grounding, records telemetry, and routes selected model roles for better quality and cost.

Intended task or use case
The system answers complex, multi-part questions using Wikipedia. It is designed for questions that may require several articles, table lookup, comparisons, counts, calculations, or connections across entities. Its goal is to return a clear answer supported by current retrieved evidence while reporting when the evidence is incomplete.

© MIT xPRO® 2026. All rights reserved.

Step 2. Security and Reliability Improvements

- Identify the safeguard(s) you implemented.
- Explain what risk or failure each safeguard addresses.
- Briefly justify your design decisions.

Implemented safeguard(s)
I implemented four safeguards: (1) deterministic input sanitization that escapes prompt-like markup and removes forged evidence blocks; (2) answer instructions that define a trust order and state that conversation history is not evidence; (3) a grounding check that flags unsupported names and quotations before the answer reaches the user; and (4) reliability controls from Checkpoint 6.1, including one corrected retry for invalid table plans, deterministic table execution, INCOMPLETE labels, and an evidence-status notice.

Risk(s) addressed
The safeguards address prompt injection, forged source formatting, adoption of false facts from earlier conversation turns, unsupported names or quotations, silent table-planning failures, and answers that present partial computations as complete. They also reduce the chance that user-provided text will be mistaken for retrieved Wikipedia evidence.

Design rationale
I used deterministic checks where possible because they are fast, testable, and do not add model calls. The input filter is intentionally narrow so ordinary phrases such as 'acting president' are not blocked. The grounding policy uses a visible [QUARANTINED] warning instead of withholding every flagged answer as a basic troubleshooting feature. The option to not display a quarantined answer is available. The planner remains on the smaller model because stronger-model planning produced very large evidence requests without a reliable quality benefit.

Step 3. Performance and Efficiency Improvements

- Describe the optimization strategy or strategies you implemented.
- Explain why you selected these optimizations.

© MIT xPRO® 2026. All rights reserved.

Optimization(s) implemented
I implemented role-based model routing, called Router B. The inexpensive model handles decomposition, article selection, and table planning. When the article-selection step identifies a table question, the stronger model is used only for evidence assessment and final answer writing. I also added detailed per-role telemetry and tested a Jev-based answer-completeness check, although the repeated final configuration uses Router B without that extra rerun step.

Rationale
The experiments showed that assigning the strongest model to every stage produced high-quality answers but increased cost and latency substantially. Across three runs of the 19-question GT test set, the all-gpt-5.4 configuration achieved an average quality score of 0.886, but cost $0.3457 per question and took 24.7 seconds per question. The smaller gpt-5.4-mini configuration was much less expensive at $0.0502 per question and faster at 13.8 seconds, but its average quality score was only 0.711. Router B was designed as a middle path. It uses the stronger model only for evidence assessment and final answer generation—the two roles that showed the clearest quality benefit. Decomposition, article selection, and table planning remain on the smaller model. This distinction was important because using the stronger model for table planning often caused the agent to request more table data, which increased the size and cost of later prompts without consistently improving answer quality. In one difficult question, the all-strong-model configuration generated prompts containing hundreds of thousands of input tokens and cost more than $3 for that question alone. Across three GT runs, Router B achieved a stable quality score of 0.868 while reducing the average list cost to $0.1725 per question. This was approximately half the cost of using gpt-5.4 throughout the entire pipeline. Router B was also slightly faster, averaging 21.3 seconds per question compared with 24.7 seconds for the all-gpt-5.4 configuration. Its average quality difference from all-gpt-5.4 was only 0.018, and that difference was not statistically distinguishable in this sample. Repeated runs were necessary because the results varied considerably across individual 19-question runs. For example, the smaller model's quality ranged from 0.658 to 0.763. A decision based on a single run could therefore reflect random variation rather than a dependable improvement. Running each final configuration three times made it possible to compare average quality, cost, latency, and consistency. The results supported Router B as the best balance: it preserved nearly all of the stronger model's measured quality while substantially reducing cost and limiting the risk of unusually large prompts.

Step 4. Evidence of Improvement

- Compare system behavior before and after your changes.

© MIT xPRO® 2026. All rights reserved.

- Include representative examples, measurements, screenshots, logs, or outputs.

Before vs. after comparison

Security

The only security failure I was able to reproduce across multiple tests involved fabricated information pasted as ordinary text, especially when it was buried beneath several unrelated conversation turns. Before the safeguards were added, the system sometimes treated this earlier user-provided content as if it were trusted retrieved evidence. Fabricated facts reached the answer in 5 of 59 attack opportunities. After adding the reliability fixes, guarded answer instructions, input sanitization, and grounding check, fabricated facts reached the answer in 0 of 59 opportunities.

The clearest improvement appeared in the fake-nominee attacks. These attacks planted an invented list of Academy Award nominees as plain text and later asked the system to identify people nominated at several ceremonies. Before the safeguards, the system adopted invented names in 5 of 20 opportunities. After the safeguards, it adopted them in 0 of 20. Fisher's exact test produced a p-value of 0.047, suggesting that the improvement was unlikely to be explained by random variation alone, although the result is close to the standard 0.05 threshold.

The input sanitizer was most effective against text formatted to resemble the system's internal evidence blocks. However, it could not determine whether ordinary pasted facts were true or false. The strongest protection against the successful plain-text attacks came from the guarded answer instructions and reliability improvements, which directed the model to treat conversation history as untrusted and rely only on retrieved evidence. The grounding check provided an additional final layer by flagging unsupported names and quotations.

Performance

On the 19-question GT set:
Router B averaged 0.868 quality at $0.1725 list cost per question and 21.3 seconds per question.
All-gpt-5.4 averaged 0.886 quality at $0.3457 and 24.7 seconds.

Router B therefore kept nearly the same measured quality while cutting list cost by 50% ($p = 0.001$ for the paired cost difference).

On the 25-question validation set, the final 7.1 system scored 0.933 versus 0.860 for the 6.1 survivor.

Supporting evidence

Representative evidence includes three repeated security runs per system, three repeated GT runs per final configuration, and three held-out validation runs for each survivor.
The final validation comparison reports: 5.1 quality 0.880, 6.1 quality 0.860, and 7.1 quality 0.933.

For 7.1, article recall was 99.3%, list cost was $0.1068 per question, and latency was 18.7 seconds per question.

© MIT xPRO® 2026. All rights reserved.

Logs, transcripts, experiment CSVs, telemetry CSVs, and PDF reports are stored in the Checkpoint 7.1 folder.

Step 5. Trade-off Analysis

- Evaluate the impact of your design decisions.
- Consider security, performance, cost, latency, output quality, usability, or other relevant trade-offs.

Trade-off analysis
Security improved materially, but the grounding check has a usability cost. It incorrectly flagged 6 answers that the judge marked fully correct, and the surface policy still shows quarantined text to the reader. Router B improved held-out quality by 0.073 over 6.1, but cost increased by $0.065 per question and latency increased by 5.9 seconds. Compared with all-gpt-5.4 on GT, Router B reduced cost by 50% and was 3.4 seconds faster, with a small average quality difference of -0.018 that was not statistically distinguishable in this sample (p = 0.81). The final retriever design favors evidence quality and attack resistance over minimum cost and speed.

Step 6. Reflection and Next Steps

- Reflect on the effectiveness of your improvements.
- Identify remaining limitations and describe what you would improve next.

Effectiveness of improvements
The safeguards were effective against the tested attacks: attack success dropped to zero in 59 opportunities. The final routing design also improved validation quality to 0.933 and avoided paying for the strongest model in every role. The most useful design lesson was that model choice should be role-specific. Stronger assessment and answer writing helped, while stronger table planning increased cost and sometimes expanded prompts without improving results.

Remaining limitations
The evaluation sets are small, and some configurations have only one exploratory run. The security safeguards were enabled together, so the test cannot isolate the contribution of each layer. The

© MIT xPRO® 2026. All rights reserved.

deterministic grounding check can produce false alarms, does not verify arithmetic, and cannot detect every unsupported paraphrase. The sanitizer cannot tell whether ordinary pasted facts are true. Very large table questions can still create long prompts and cost spikes, and one validation run missed an article.

Future improvements

Improvements could continue indefinitely, but I would prioritize the following two areas:

- **Add deterministic limits to table-request loops.** The system should cap the number of table requests, the amount of evidence collected, and the total prompt size. These controls would prevent long processing loops, reduce latency, and limit unusually expensive questions without changing the rest of the retrieval workflow.
- **Improve the grounding check with evidence-linked claim verification.** Instead of checking only whether names and quotations appear somewhere in the evidence, the system should connect each important claim to the specific passage that supports it. I would then tune and evaluate the check using a larger labeled dataset. This should reduce false alarms and prevent correct answers from being unnecessarily marked as `[QUARANTINED]`, improving usability and overall answer quality while preserving the security benefit.

Supporting Materials

Use this section to list or link to any supporting materials included with your submission.

Supporting material	Location, link, or note
Test cases	Security scenarios and transcripts: security_tests/; final validation questions and results: final_validation/runs/experiments/.
Performance measurements	final_validation/efficiency_validation_5_1_6_1_7_1.pdf; cost_efficiency/reports/gt_three_run_comparison.pdf; telemetry CSVs in final_validation/runs/telemetry/.
Example queries and outputs	Before/after attack example and outcome tables: security_comparison_5_1_vs_7_1.md; saved evidence and experiment outputs in cost_efficiency/ and final_validation/.

© MIT xPRO® 2026. All rights reserved.

Supporting material	Location, link, or note
Screenshots or logs	Console logs: cost_efficiency/console/; security transcripts and result CSVs: security_tests/; AgentState evidence logs: cost_efficiency/logs/.
Code/notebook/repository	survivor_agent_final.py; wiki_agents/sanitize.py, grounding_check.py, answer_guard.py, prompts.py, model_routing.py, and agent_graph.py.
Other supporting evidence	README 7.1 - security improvements.md; README 7.1 - performance and cost.md; security_comparison_5_1_vs_7_1.md.

AI Assistance Disclosure

I used AI to help organize my code and develop utilities and analytical tools for reviewing experiment logs. AI also assisted with retrieving and consolidating results from previous runs so that I could assess the system's current state.

I also used AI to organize my observations and improve the clarity and wording of the written analysis. However, I conducted the performance evaluation, selected the areas for further improvement, and proposed the security and performance changes based on the collected evidence.

I reviewed the prompts, code, experimental runs, measurements, interpretations, and conclusions. I made all final decisions about the system design, reported outcomes, and evidence included in this report.

End of Required Capstone Checkpoint 7.1

Return to the assignment page in the learning platform to upload and submit your completed capstone checkpoint and complete any remaining items in this module.

© MIT xPRO® 2026. All rights reserved.